GAME PROGRAMMING COURSE — HARDWARE

PCパーツの仕組み

CPU・GPU・メモリから、ゲームが動くまでの全体像

全3時間

PC / PARTS

LESSON 1

PCの全体像とCPU・メモリ

目標：プログラムが CPU とメモリの上でどう動いているかを説明できる

全体像

PCを作っているパーツ

パーツ	役割
CPU	プログラムの命令を読み、計算と判断をする
メモリ（RAM）	今使っているデータを一時的に置く
ストレージ	電源を切っても消えないようにデータを保管する
GPU	画面に出す絵を、大量の計算で一気に描く
マザーボード	すべてのパーツをつなぎ、データの通り道になる
電源ユニット	コンセントの電気を、パーツが使える形に変える
冷却・ケース	パーツが出す熱を外へ逃がす
入出力機器	キーボード・マウス・モニター・スピーカーなど

POINT

3時間で、この表のパーツを1つずつ「中で何が起きているか」まで掘り下げます。最後に、ゲームを作る立場から見た PC の選び方につなげます。

全体像

コンピュータの5大装置



- **プログラム内蔵方式**：プログラムもデータと同じように記憶装置に置き、CPU が1つずつ読んで実行する
- 考えたのは数学者フォン・ノイマンら（1945年ごろ）。今の PC もスマホもゲーム機も、この形
- CPU と記憶装置の間の行き来が多く、ここが速さの壁になりやすい

全体像

ゲームを起動すると、中で何が起きるか

1	2	3	4	5
ストレージ	**メモリ**	**CPU**	**GPU**	**モニター**
ゲームの実行ファイルとデータを読み出す	読み出したものを置く。ここが「ロード」	入力・物理・AI などゲームの処理を毎フレーム計算	CPU の指示を受けて、その瞬間の絵を描く	描き終わった絵を表示する

- 3〜5 を、**1秒間に60回以上**くり返しているのがゲーム
- どこか1つが遅いと、全体がそこで詰まる（これを「ボトルネック」と呼ぶ）
- この間ずっと、マザーボードが道をつなぎ、電源が電気を送り、冷却が熱を逃がしている

POINT

Unity の Update() が1フレームに1回呼ばれるのは、この「3」の部分。今日の話はすべて、ふだん書いているコードの**下で起きていること**です。

基礎

データの単位：ビットとバイト

単位	大きさ
1 bit（ビット）	0 か 1 の1けた。電気が「ある・ない」
1 byte（バイト）	8 bit。256通りを表せる。半角1文字ぶん
1 KB（キロ）	約1,000 byte。短い文章
1 MB（メガ）	約100万 byte。写真1枚ほど
1 GB（ギガ）	約10億 byte。メモリやゲームの容量
1 TB（テラ）	約1兆 byte。ストレージの容量

1TB の SSD が 931GB？

メーカーは **1KB＝1,000** で数える。Windows は **1KB＝1,024**（2の10乗）で数える。

1,000,000,000,000 ÷ 1,073,741,824
≒ 931

容量が減ったのではなく、**数え方が違うだけ**。なぜ2通りあるのかは次のページで。

POINT 通信速度は**ビット**（Mbps）、容量は**バイト**（MB）で表すことが多い。1 Gbps の回線でも、1秒に届くのは最大で約 125 MB です。

基礎

なぜ「1KB」に2通りの意味があるのか

1KB＝1,024（コンピュータの慣習）

- コンピュータは**2進数**で場所（番地）を数える
- だからメモリの容量は自然に**2のべき乗**になる（8GB のメモリは 2の33乗バイト）
- 2の10乗＝1,024 が 1,000 に近いので、昔から「キロ」と呼んでしまった

1KB＝1,000（国際単位系）

- キロ・メガ・ギガは、本来 **10進数**の決まり（1km＝1,000m）
- ストレージのメーカーや通信速度は、こちらで表示する
- 単位の決まりとしては、こちらが正しい使い方

混乱を解くため、1998年に国際的な規格団体（IEC）が **2進数専用の単位** を作った。

10進（SI）	大きさ	2進（IEC）	大きさ
kB（キロ）	1,000	KiB（キビ）	1,024
MB（メガ）	1,000,000	MiB（メビ）	1,048,576
GB（ギガ）	1,000,000,000	GiB（ギビ）	1,073,741,824
TB（テラ）	1,000,000,000,000	TiB（テビ）	1,099,511,627,776

基礎

同じ 1TB でも、表示は OS によって違う

OS	数え方	1TB の表示
Windows	2進で計算し、単位は「GB」と書く	931 GB
macOS	10進で計算（2009年から）	1 TB
Linux	ソフトによる。GiB と正しく書くものも	931 GiB など

箱の表記	Windows の表示
500 GB	465 GB
1 TB	931 GB
2 TB	1.81 TB
4 TB	3.63 TB

単位が大きいほどズレも大きい

KB で約 2%、MB で約 5%、GB で約 7%、**TB では約 9%** 小さく見える。

実際はさらに少し減る

OS の起動用や回復用の領域、ファイルの管理情報が場所を使う。

メモリは最初から2進

16GB のメモリは本当に 16GiB。だから Windows でも 16GB と表示される。

POINT

ゲームの「必要な空き容量」やセーブデータの大きさを画面に出すときは、**1000 と 1024 のどちらで割るか**を決めてそろえる。遊ぶ人の OS の表示とずれると問い合わせの元になる。

見てみよう

自分の PC の中身を確かめる

Ctrl + Shift + Esc でタスクマネージャーを開き、**「パフォーマンス」**タブを見る。

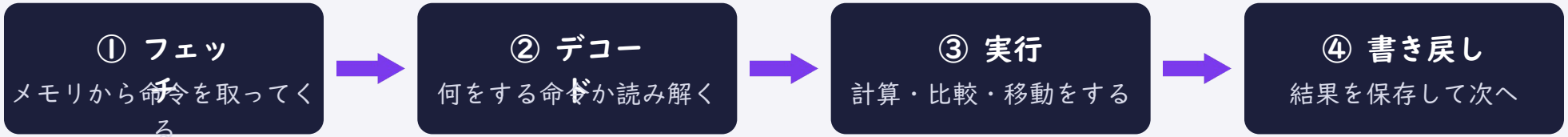
項目	見るところ	この授業で出てくる話
CPU	名前、速度、コア数、論理プロセッサ数、L1/L2/L3 キャッシュ	クロック、コアとスレッド、キャッシュ
メモリ	容量、速度、使用中のスロット数	DDR の速度、デュアルチャネル
ディスク	種類（SSD / HDD）、読み書きの速さ	HDD と SSD、NVMe
GPU	名前、専用 GPU メモリ、共有 GPU メモリ	VRAM、内蔵 GPU
Wi-Fi / イーサネット	送受信の速さ	ネットワークの遅延と帯域

POINT

ここに並ぶ数字を、今日の授業で1つずつ読めるようにします。気になる数字をメモしておき、あとで「これのことか」と照らし合わせよう。

CPU

CPU は命令を1つずつ処理する



制御装置

命令を読み解き、各部に指示を出す

演算装置（ALU）

足し算・比較などを実際に行う

レジスタ

CPU の中の、一番速くて小さい置き場

POINT

C# の `hp -= damage;` の1行も、CPU にとっては「読み込む・引く・書き戻す」などの**小さな命令の並び**に変換されて実行されている。

CPU

クロック周波数：CPU の心拍数

4.0 GHz

＝ 1秒間に 40億回

1回あたり 0.25 ナノ秒

- CPU は「チク、タク」という信号（クロック）に合わせて動く
- Hz は1秒間の回数。G（ギガ）は10億
- 0.25 ナノ秒の間に、光でさえ **7.5cm** しか進めない
- **ベースクロック**：いつでも保証される速さ
- **ブーストクロック**：温度と電力に余裕があるときだけ出す最高速

注意 クロックが高い＝速い、とは限らない。**10年前の 4GHz の CPU は、今の 4GHz の CPU よりずっと遅い**。理由は次のページ。

CPU の速さを決める3つの要素

速さ ≒ クロック × IPC × 使えるコア数

クロック

足を動かす**回数**。上げると熱と電力が急に増えるので、ここ20年ほどは 5GHz 前後で頭打ち。

IPC

1クロックで進める命令の数＝**歩幅**。世代が新しくなるたびに、設計の工夫で少しずつ伸びている。

コア数

走る**人数**。ただし、仕事を分けられる処理でしか効かない。

POINT クロックが伸びなくなってから、CPU は**歩幅（IPC）と人数（コア数）**で速くなってきた。だからカタログの GHz だけで比べてはいけない。

CPU

コアとスレッド

コア（物理的な数）

- 1つの CPU の中にある、独立した処理装置
- 8コアなら、8つの仕事を本当に同時に進められる

スレッド（論理的な数）

- 1コアを2つあるように見せる技術（SMT、ハイパースレッディング）
- 片方が待っている間に、もう片方が空いた部品を使う。**2倍にはならない**

Unity の中のスレッド

メインスレッド：Update() や MonoBehaviour の処理。**レンダースレッド**：GPU への描画命令。**ワーカースレッド**：Job System や物理演算の一部。

POINT

ふつうに書いた C# スクリプトは**ほぼメインスレッドの1本**で動く。だからゲームでは、コアの多さより**1コアあたりの速さ**が効く場面が多い。

CPU

大きいコアと小さいコアの組み合わせ

最近の CPU は、すべてのコアが同じとは限らない。**性能重視のコア**と**省電力重視のコア**を混ぜて載せる。

種類	特徴	向いている仕事
P コア（Performance）	大きく、速く、電力を多く使う	ゲームのメインスレッド、重い計算
E コア（Efficient）	小さく、省電力。同じ面積にたくさん載る	裏で動くアプリ、並列にできる大量の処理

- Intel は第12世代（2021年）からこの形。「24コア（8P＋16E）」のように表記される
- どの仕事をどのコアに回すかは、**OS（Windows）が自動で決める**
- スマホの CPU は、もっと前からこの方式（大きいコア＋小さいコア）

注意 「24コア」と書いてあっても、24個とも速いわけではない。**コア数の合計だけで比べない**。ゲームの速さに効くのは主に P コアの数と速さ。

CPU の工夫

パイプライン：流れ作業で歩幅を広げる

洗濯を「洗う・乾かす・たたむ」に分ける。1回分が終わるのを待たずに、**次の洗濯物を洗い始める**。

工程 ＼ 時間	1	2	3	4	5
洗う	A	B	C		
乾かす		A	B	C	
たたむ			A	B	C

1つずつなら 9 かかる作業が **5** で終わる。CPU は命令の処理を十数段に分けて、これを行っている。

弱点

if 文があると、どちらに進むか決まるまで次を始められない。そこで CPU は行き先を**予想して先に進める（分岐予測）**。外れると途中までの作業を捨ててやり直す。

CPU の工夫

キャッシュ：手元に置いて待ち時間を減らす

メモリは CPU から見ると**とても遠い**。よく使うデータを、CPU の中の小さく速い置き場に写しておく。

段	容量の目安	待ち時間の目安	たとえ
L1	コアごとに数十 KB	約 1 ナノ秒	手に持っているメモ
L2	コアごとに 1〜3 MB	約 3〜4 ナノ秒	机の引き出し
L3	全コアで共有、数十 MB	約 10 ナノ秒	部屋の本棚
メモリ	数十 GB	約 80〜100 ナノ秒	隣の建物の図書館

- 欲しいデータがキャッシュにある＝**ヒット**、ない＝**ミス**（メモリまで取りに行く）
- L3 を縦に積み重ねて増やした CPU（AMD の「X3D」）は、ゲームで特に速い

記憶の階層

L1 が「1秒」だったら、ほかはどれくらいか

置き場所	実際の待ち時間（目安）	L1 を 1 秒としたとき
L1 キャッシュ	約 1 ナノ秒	1 秒
L3 キャッシュ	約 10 ナノ秒	10 秒
メモリ	約 100 ナノ秒	約 1 分 40 秒
SSD（NVMe）	約 100 マイクロ秒	約 1 日
HDD	約 10 ミリ秒	約 4 か月

上に行くほど

速い・小さい・1GB あたりが高い

下に行くほど

遅い・大きい・安い。電源を切っても消えない

プログラムとの関係

キャッシュに優しいコード

C# Grid.cs

```
int[,] map = new int[1000, 1000];

// 速い：となりのデータを順に読む
for (y = 0; y < 1000; y++)
  for (x = 0; x < 1000; x++) sum += map[y, x];

// 遅い：飛び飛びに読む
for (x = 0; x < 1000; x++)
  for (y = 0; y < 1000; y++) sum += map[y, x];
```

- キャッシュは**64バイトずつまとめて**運ばれる
- となりのデータを続けて使うと、運んだぶんが無駄にならない（空間的局所性）
- 同じデータを何度も使うと、手元に残っていて速い（時間的局所性）

POINT

2つのループは計算の量が同じなのに、速さが何倍も違うことがある。Unity の **DOTS（ECS）**は、データを並べ方から設計してキャッシュを効かせる仕組み。

CPU の種類

命令セット：x86-64 と ARM

CPU が理解できる命令の「言葉」を**命令セット**と呼ぶ。言葉が違えば、同じプログラムはそのまま動かない。

項目	x86-64	ARM（ARM64）
主なメーカー	Intel、AMD	Apple、Qualcomm など（設計図を Arm 社が提供）
主な使われ方	Windows の PC、PS5、Xbox Series X|S	スマホ、Mac、Nintendo Switch、一部の Windows ノート
得意なこと	高い性能。長年のソフト資産	少ない電力で高い効率

POINT

Unity で Android や iOS 向けにビルドすると、同じ C# から **ARM の機械語**が作られる。Windows 向けなら x86-64。ビルド設定の「Architecture」の項目はこの選択です。

CPU の選び方

型番の読み方

Intel の例

Core Ultra 7 265K

7＝グレード（5 / 7 / 9）、2＝世代、65＝同世代内の位置、K＝末尾の記号

AMD の例

Ryzen 7 9800X3D

7＝グレード（5 / 7 / 9）、9＝世代、800＝同世代内の位置、X3D＝末尾の記号

記号	メーカー	意味
K	Intel	クロックの上限が解放されている（オーバークロックできる）
F	Intel	内蔵 GPU がない。グラフィックボードが必須
X	AMD	クロックを高めに設定した上位版
X3D	AMD	L3 キャッシュを積み増した版。ゲームに強い
G	AMD	強めの内蔵 GPU を持つ版

CPU の中の GPU

内蔵 GPU と単体 GPU

内蔵 GPU（iGPU）

- CPU のチップの中に入っている GPU
- 専用のメモリを持たず、**メインメモリを借りて使う**
- 省電力・安い。事務や動画、軽いゲーム向け

単体 GPU（dGPU）

- グラフィックボードとして別に挿す GPU
- 専用の高速なメモリ（VRAM）を持つ
- 電力と熱は大きいが、性能は桁違い

注意 モニターのケーブルを**マザーボード側の端子**に挿すと、グラフィックボードがあっても内蔵 GPU で描画されて遅くなる（または何も映らない）。ケーブルはグラフィックボードの端子へ。

POINT タスクマネージャーの GPU 欄に2つ出る PC は、両方を持っている。ノート PC は場面によって自動で切り替えている。

メモリ

メモリはなぜ電源を切ると消えるのか

DRAM（メインメモリ）

- 小さな**コンデンサに電気をためて** 1 と 0 を表す
- 電気は少しずつ漏れるので、**1秒に何十回も読み直して**書き直す（リフレッシュ）
- 電源が切れると、ためた電気がなくなり消える
- 構造が単純なので、大容量を安く作れる

SRAM（キャッシュ）

- スイッチ（トランジスタ）6個ほどで 1bit を保つ
- リフレッシュが要らず、とても速い
- 1bit に使う面積が大きく、大容量にできない
- これも電源を切ると消える

POINT 電源を切っても消えない記憶（ストレージ）は、2時間目に扱う。ゲームの**セーブ**とは、メモリ上の状態をストレージに書き出すことです。

メモリ

DDR4 と DDR5：「DDR5-6000」の意味

項目	DDR4	DDR5
登場	2014年ごろ	2021年ごろ
よくある速度	DDR4-3200	DDR5-5600〜6400
電圧（標準）	1.2 V	1.1 V
互換性	切り欠きの位置が違う	相互に挿せない

速さを計算してみる

6000 ＝ 1秒に **60億回** 転送

1回に運べるのは **8 バイト**

60億 × 8 ＝ 48 GB/秒

POINT

DDR は Double Data Rate の略。時計の信号が**上がるときと下がるときの両方**でデータを送るので、実際のクロック（3000MHz）の2倍の回数になる。

意識しないと損する

デュアルチャネルと XMP / EXPO

2枚組で、道を2本にする

CPU とメモリの間には通り道（チャネル）が2本ある。**2枚に分けて挿すと、両方を同時に使える。**

挿し方	DDR5-6000 の場合
16GB × 1枚	約 48 GB/秒
8GB × 2枚	約 96 GB/秒

箱の速度は、設定しないと出ない

「DDR5-6000」のメモリも、挿しただけでは**安全な低めの速度**で動く。BIOS で **XMP**（Intel）や **EXPO**（AMD）を有効にすると、表示どおりの速度になる。

注意 4本のスロットに2枚挿すときは、**決められた位置**（多くは A2 と B2）に挿す。隣どうしに挿すと、同じ道に2枚並んで効果がない。マザーボードの説明書で確認する。

メモリ

容量が足りなくなると何が起きるか

容量	目安（2026年時点）
8 GB	ブラウザで多くのタブを開くだけで足りなくなることがある
16 GB	ゲームを遊ぶための最低ライン
32 GB	Unity での開発、ゲーム＋配信、ブラウザを開きながらの作業に安心
64 GB 以上	大規模な 3D 制作、動画編集、たくさんのツールの同時使用



POINT

容量は「足りているか、いないか」の問題。**足りていれば、増やしても速くならない**。足りないと、記憶の階層を何段も下りるので一気に遅くなる。